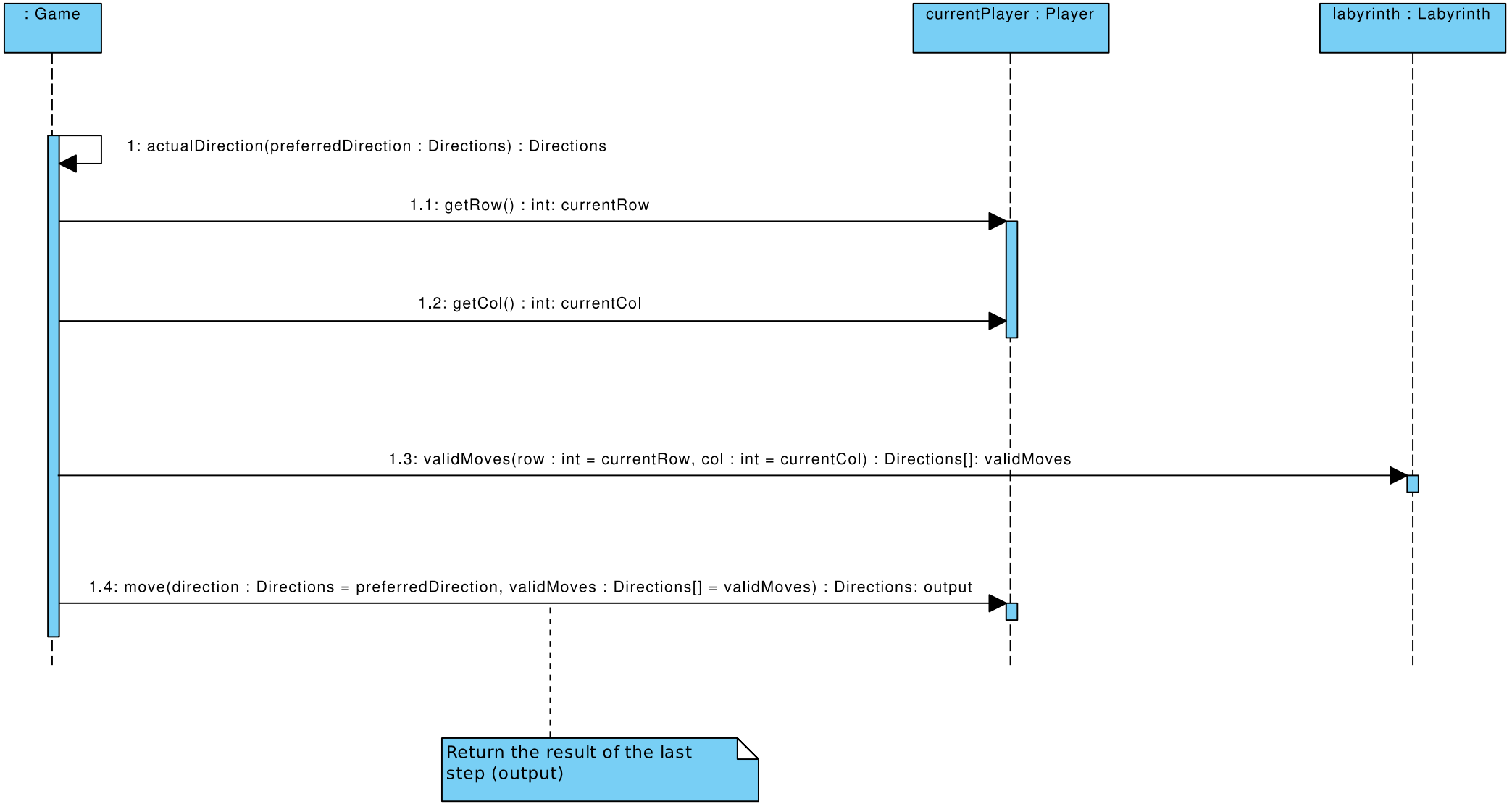
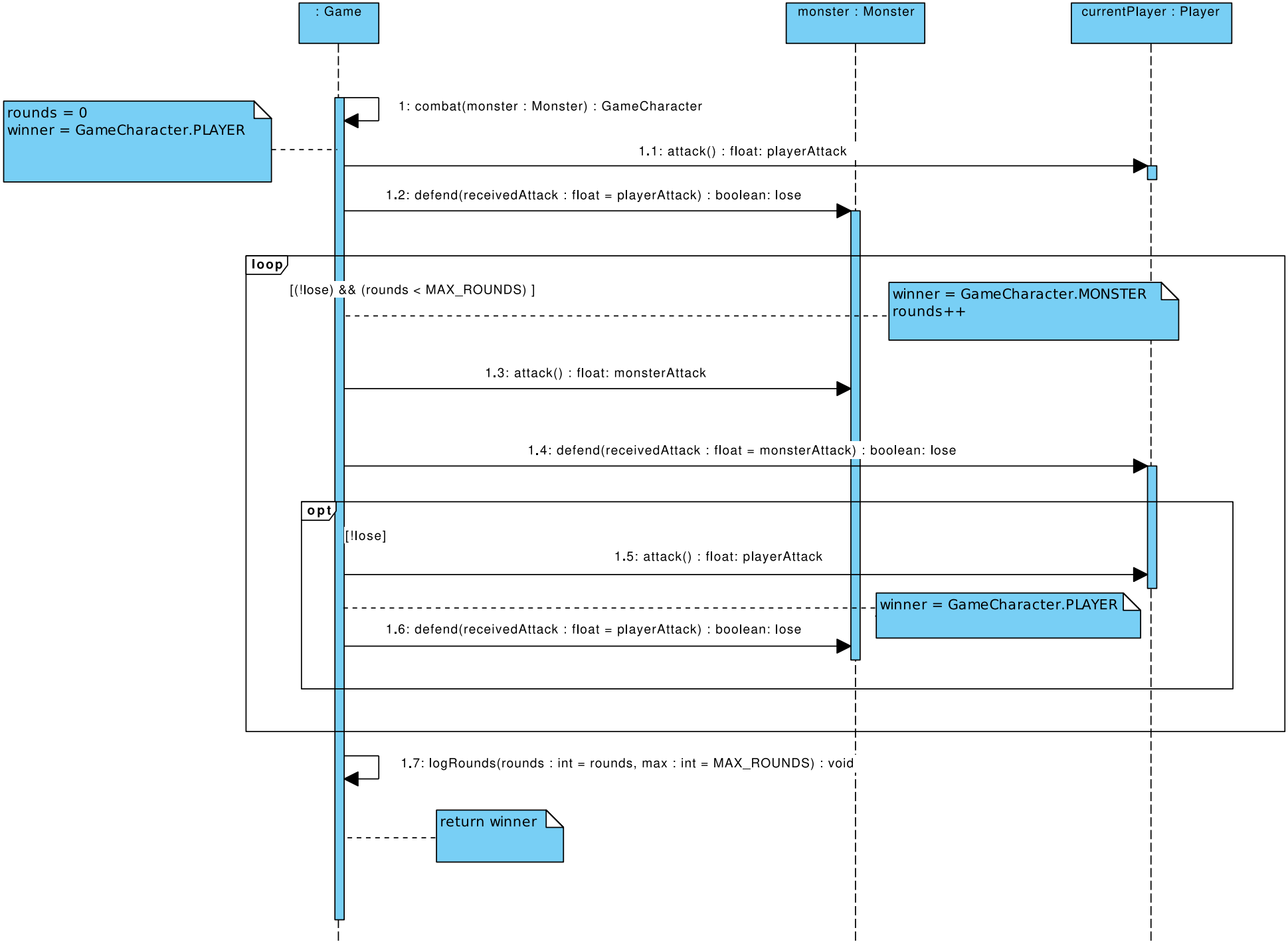
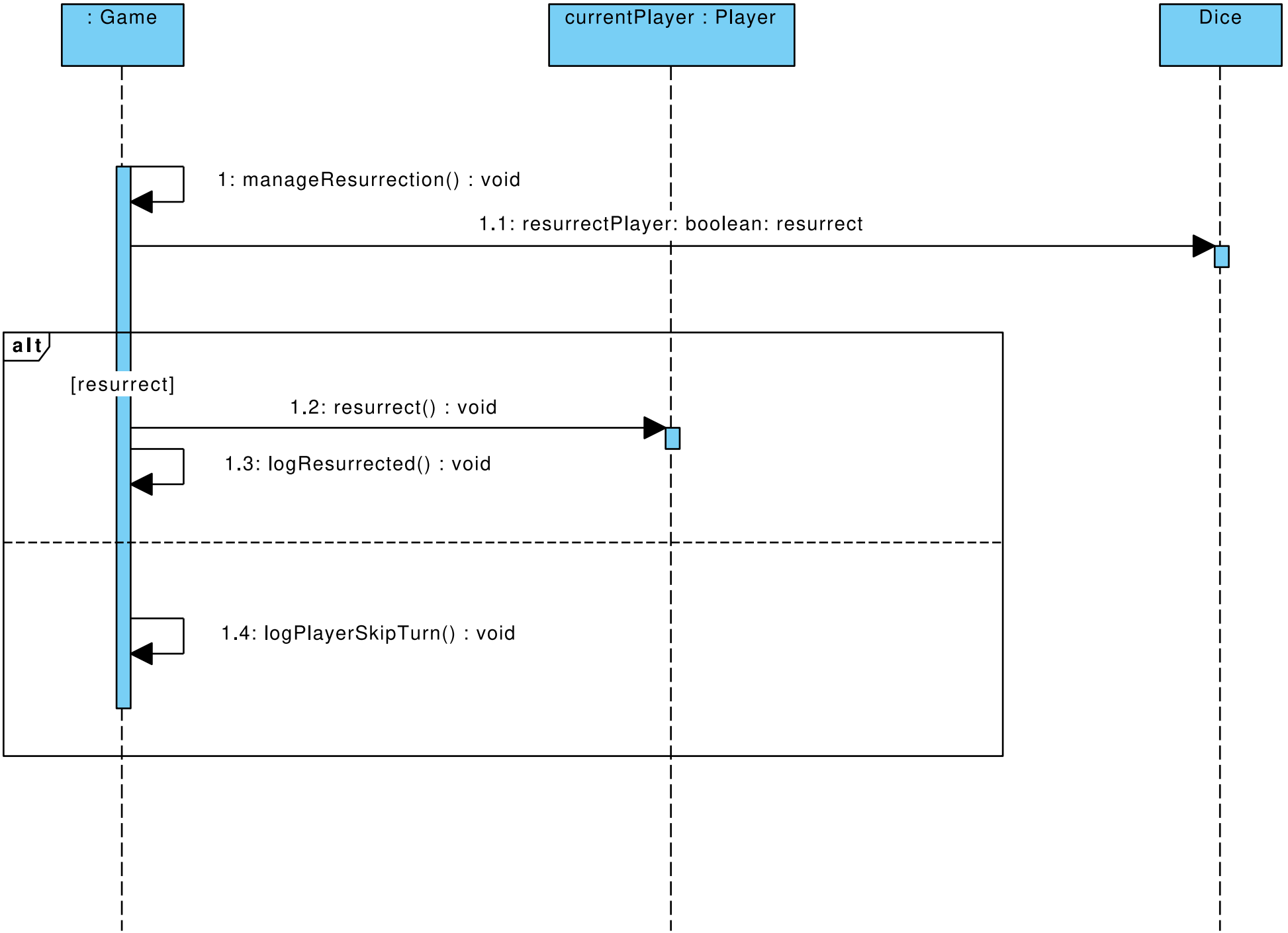
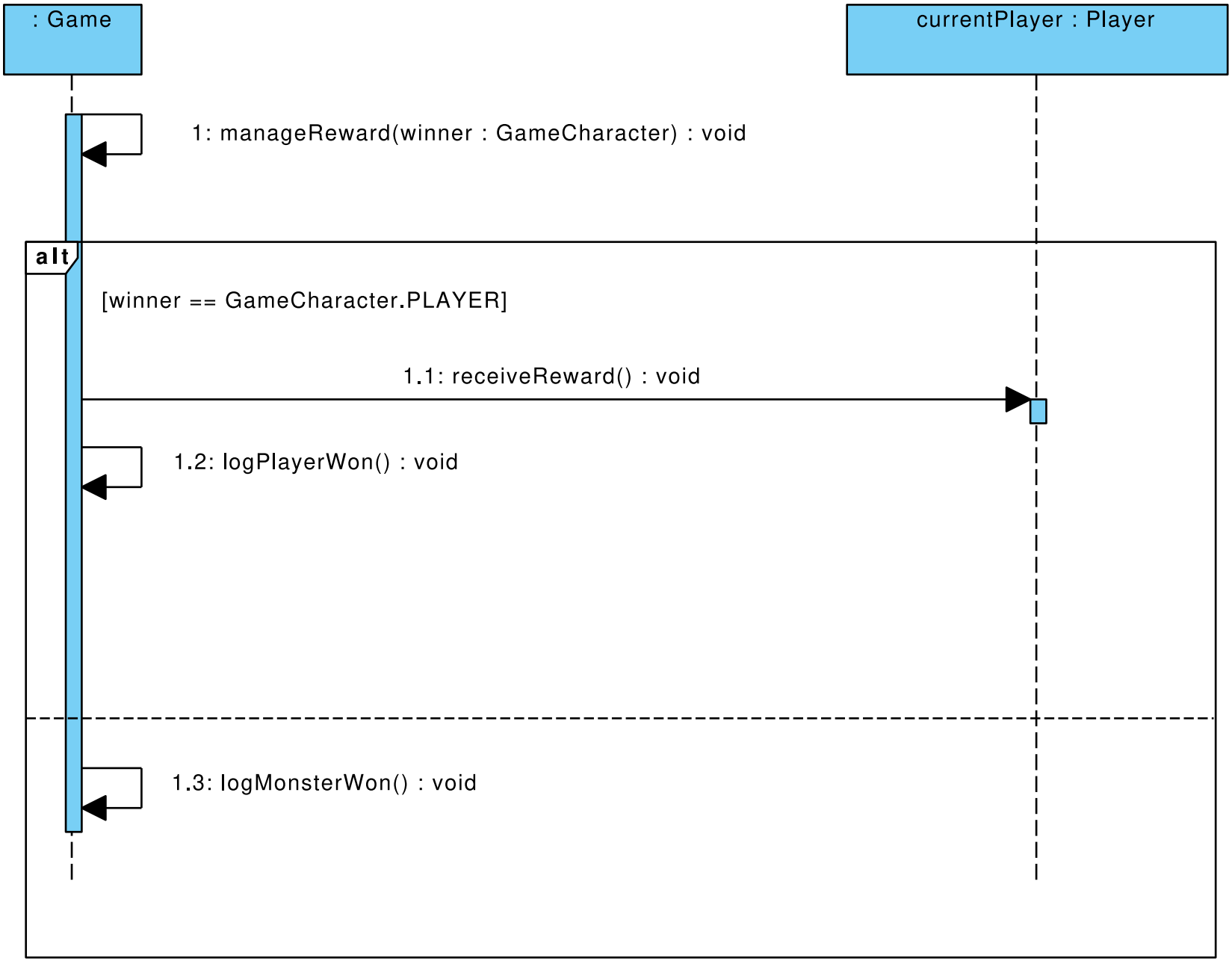
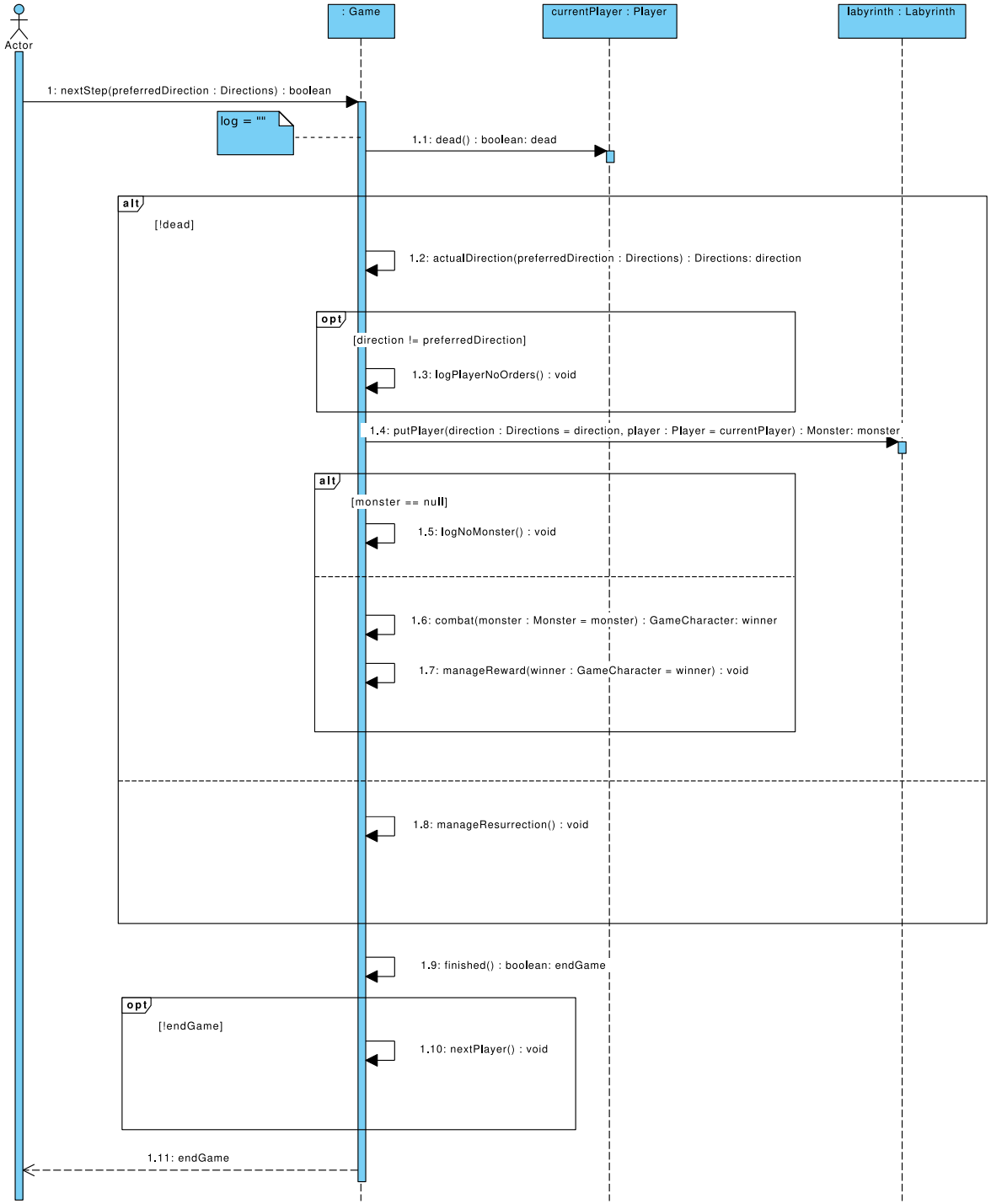
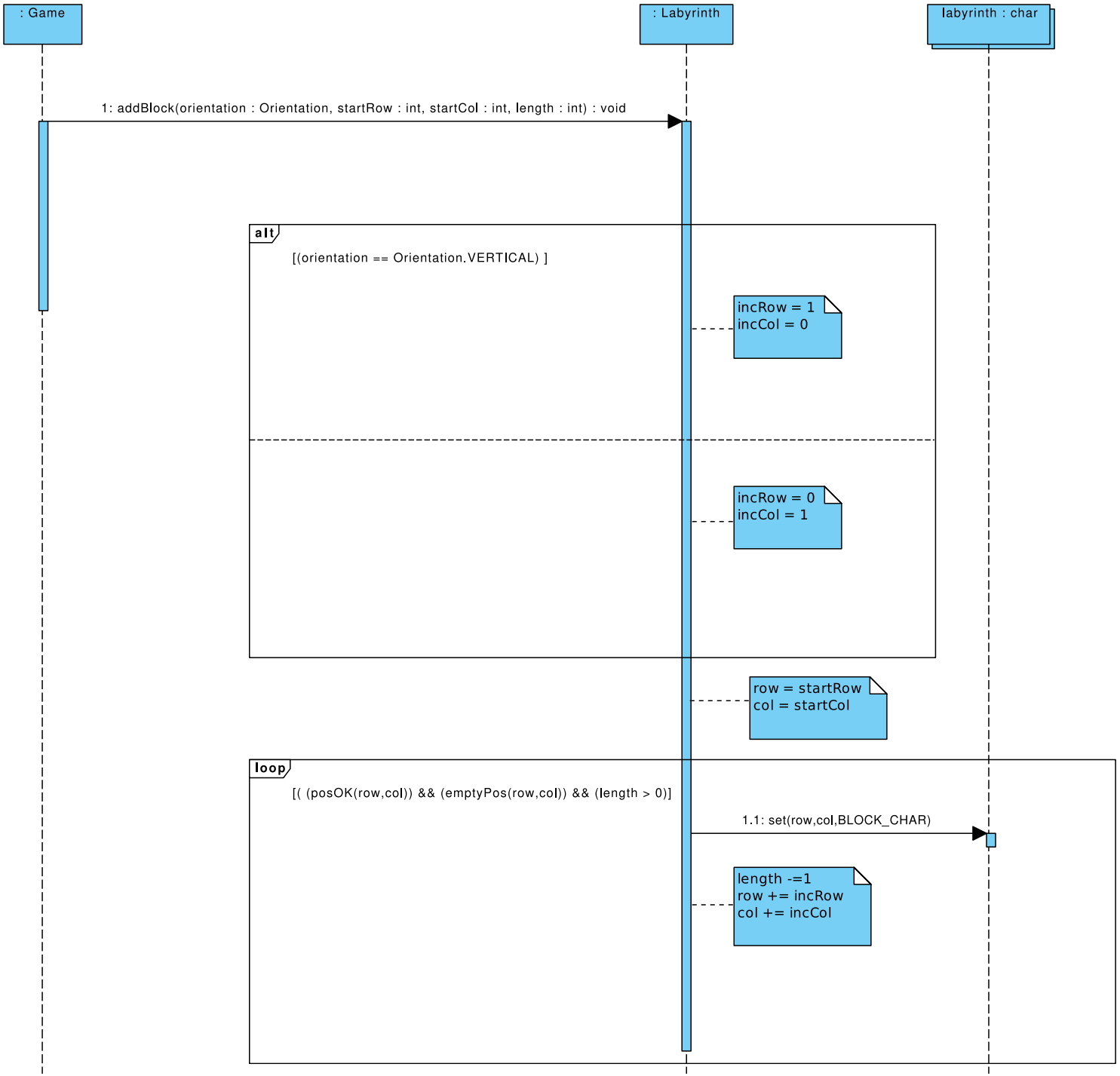

: Game
currentPlayer : Player
labyrinth : Labyrinth
1: actualDirection(preferredDirection : Directions) : Directions
1.1: getRow() : int: currentRow
1.2: getCol() : int: currentCol
1.3: validMoves(row : int = currentRow, col : int = currentCol) : Directions[]: validMoves
1.4: move(direction : Directions = preferredDirection, validMoves : Directions[] = validMoves) : Directions: output
Return the result of the last step (output)

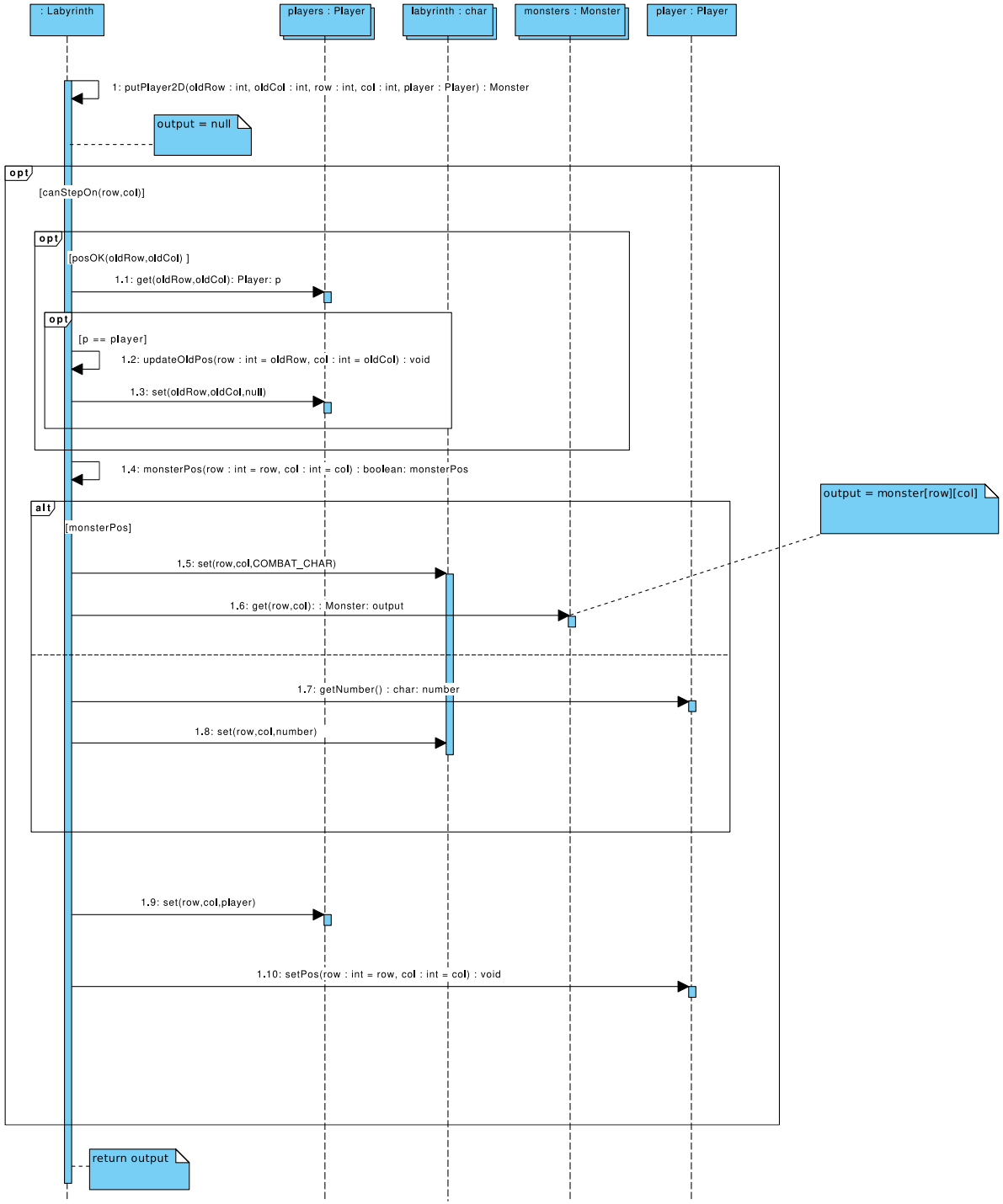

: Game
monster : Monster
currentPlayer : Player
rounds = 0
winner = GameCharacter.PLAYER
1: combat(monster : Monster) : GameCharacter
1.1: attack() : float: playerAttack
1.2: defend(receivedAttack : float = playerAttack) : boolean: lose
loop
[(!lose) && (rounds < MAX_ROUNDS)]
winner = GameCharacter.MONSTER
rounds++
1.3: attack() : float: monsterAttack
1.4: defend(receivedAttack : float = monsterAttack) : boolean: lose
opt
[!lose]
1.5: attack() : float: playerAttack
winner = GameCharacter.PLAYER
1.6: defend(receivedAttack : float = playerAttack) : boolean: lose
1.7: logRounds(rounds : int = rounds, max : int = MAX_ROUNDS) : void
return winner

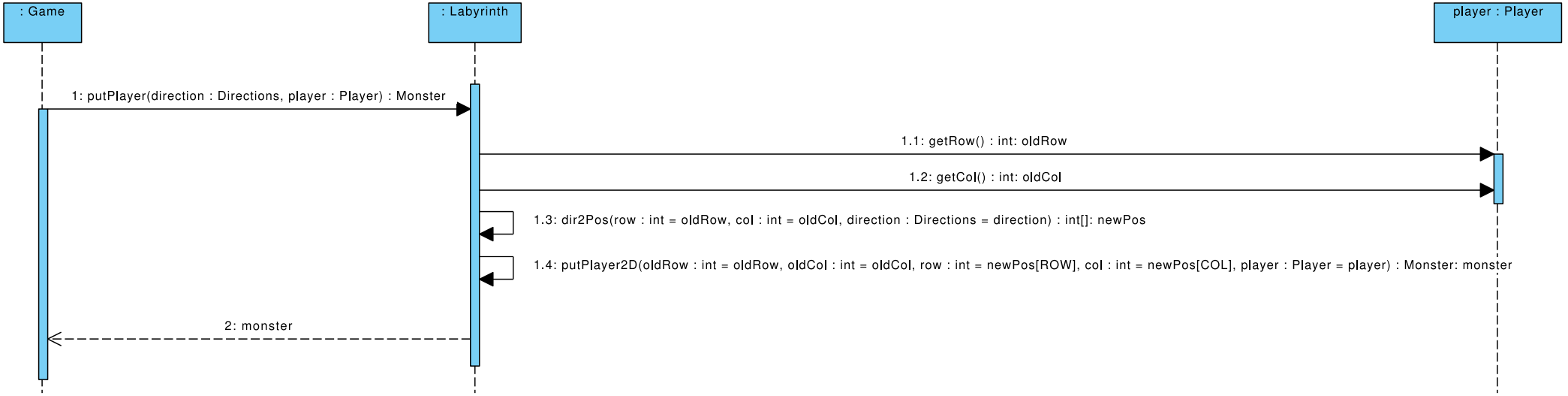

: Game
currentPlayer : Player
Dice
1: manageResurrection() : void
1.1: resurrectPlayer: boolean: resurrect
alt
[resurrect]
1.2: resurrect() : void
1.3: logResurrected() : void
1.4: logPlayerSkipTurn() : void

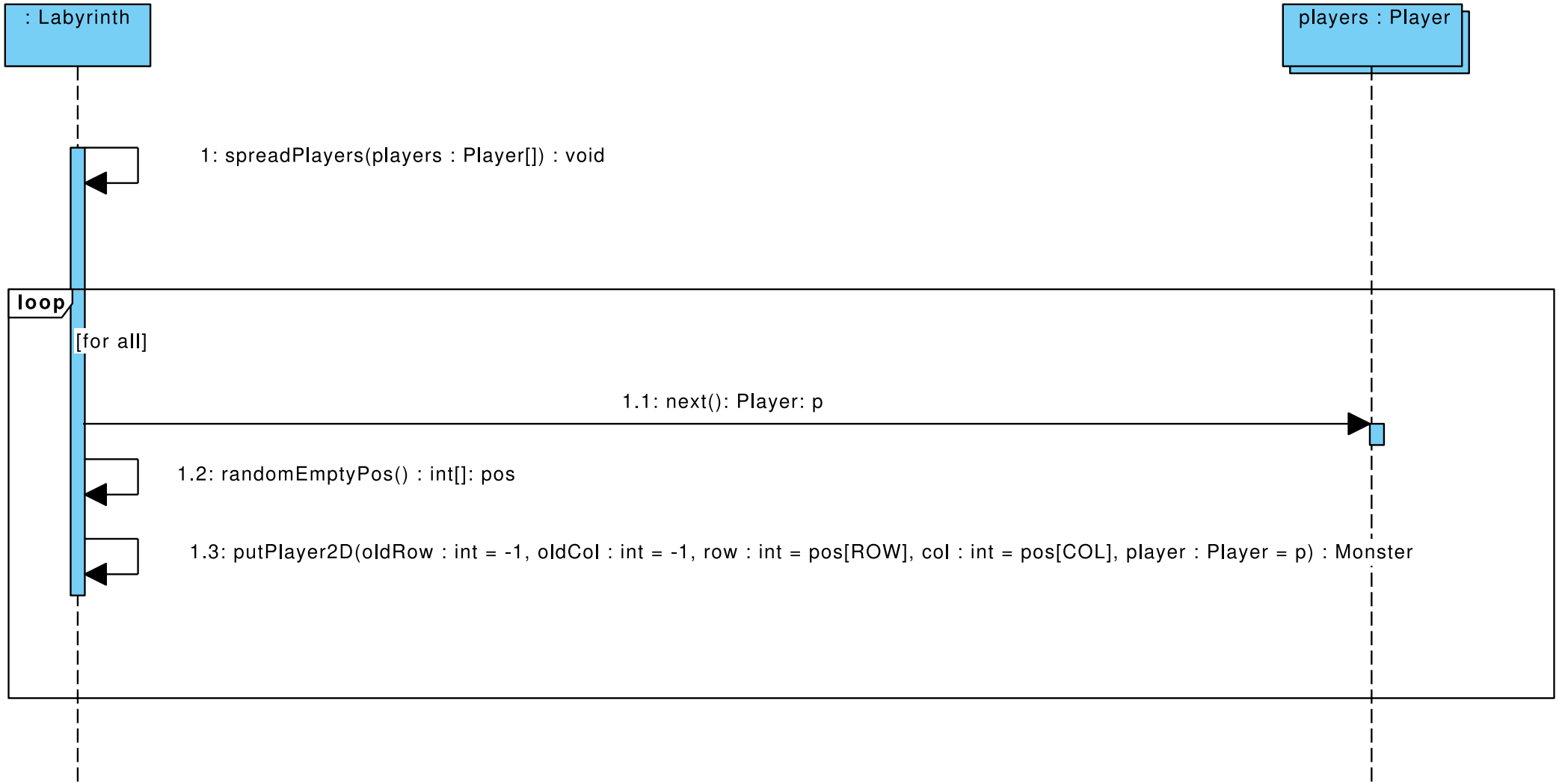

: Game
currentPlayer : Player
1: manageReward(winner : GameCharacter) : void
alt
[winner == GameCharacter.PLAYER]
1.1: receiveReward() : void
1.2: logPlayerWon() : void
1.3: logMonsterWon() : void

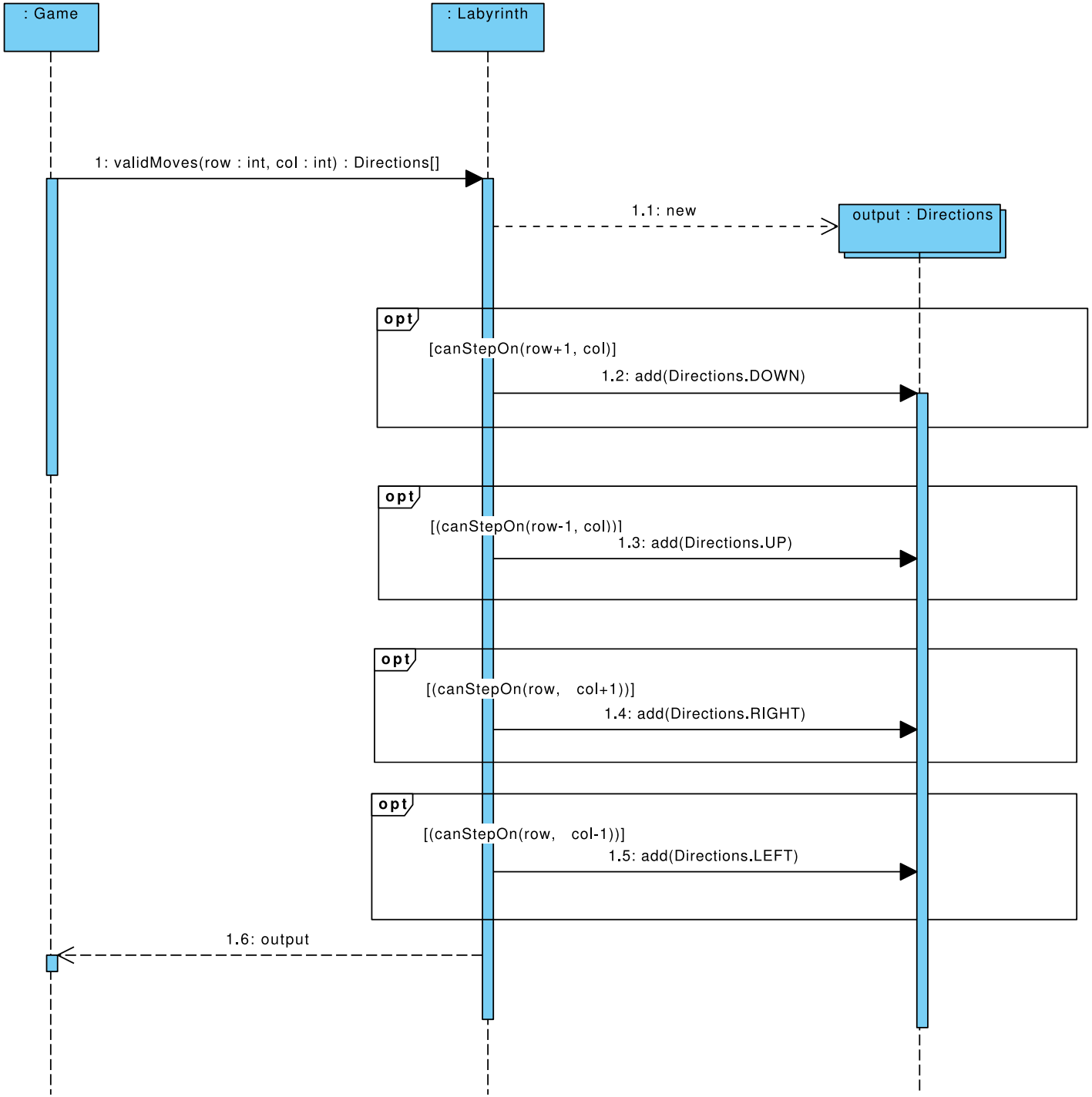

Actor
: Game
currentPlayer : Player
labyrinth : Labyrinth
1: nextStep(preferredDirection : Directions) : boolean
log = ""
1.1: dead() : boolean: dead
alt
[!dead]
1.2: actualDirection(preferredDirection : Directions) : Directions: direction
opt
[direction != preferredDirection]
1.3: logPlayerNoOrders() : void
1.4: putPlayer(direction : Directions = direction, player : Player = currentPlayer) : Monster: monster
alt
[monster == null]
1.5: logNoMonster() : void
1.6: combat(monster : Monster = monster) : GameCharacter: winner
1.7: manageReward(winner : GameCharacter = winner) : void
1.8: manageResurrection() : void
1.9: finished() : boolean: endGame
opt
[!endGame]
1.10: nextPlayer() : void
1.11: endGame

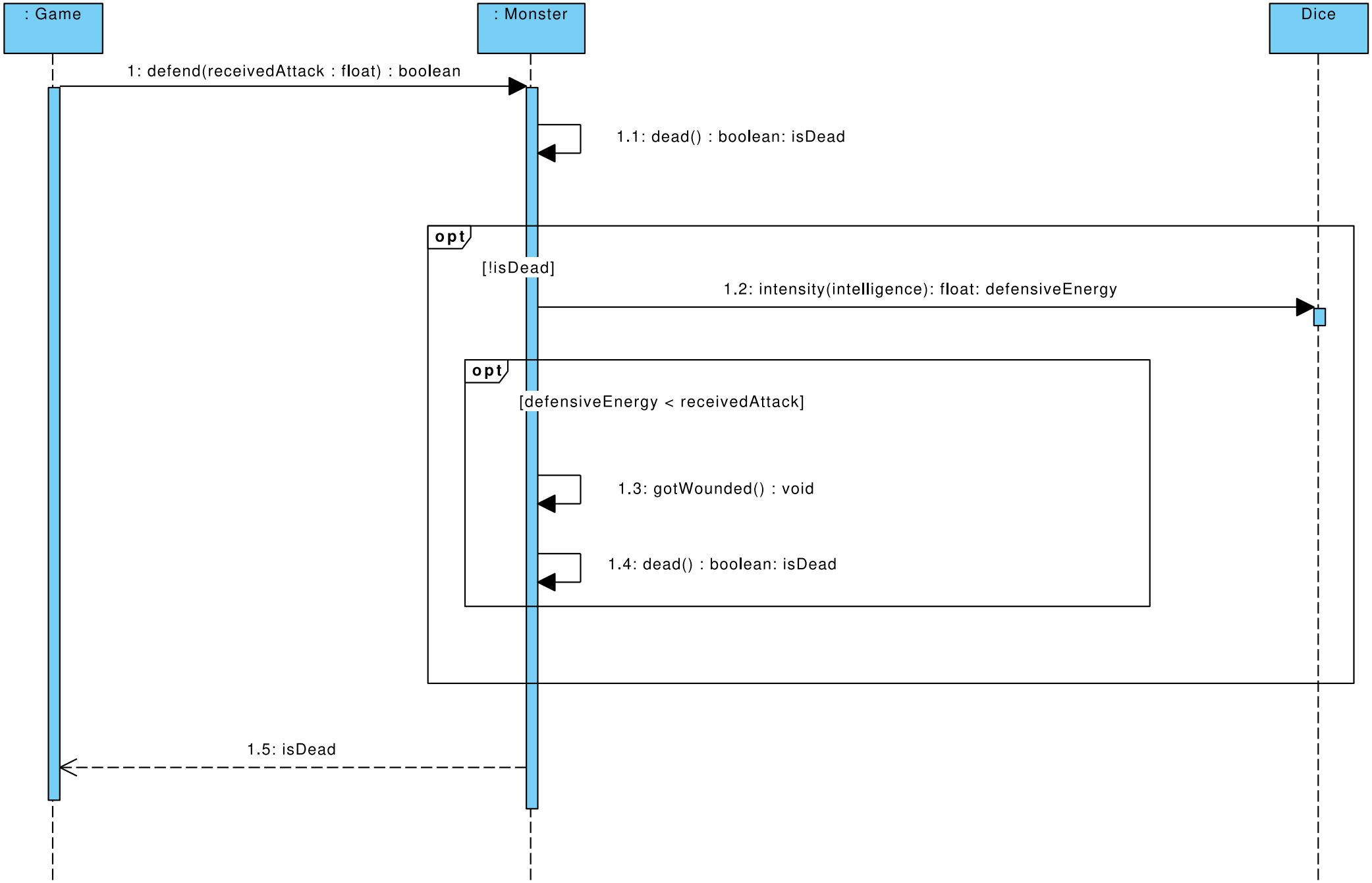

: Game
: Labyrinth
labyrinth : char
1: addBlock(orientation : Orientation, startRow : int, startCol : int, length : int) : void
alt
[(orientation == Orientation.VERTICAL)]
incRow = 1
incCol = 0
incRow = 0
incCol = 1
row = startRow
col = startCol
loop
[((posOK(row,col)) && (emptyPos(row,col)) && (length > 0)]
1.1: set(row,col,BLOCK_CHAR)
length -=1
row += incRow
col += incCol

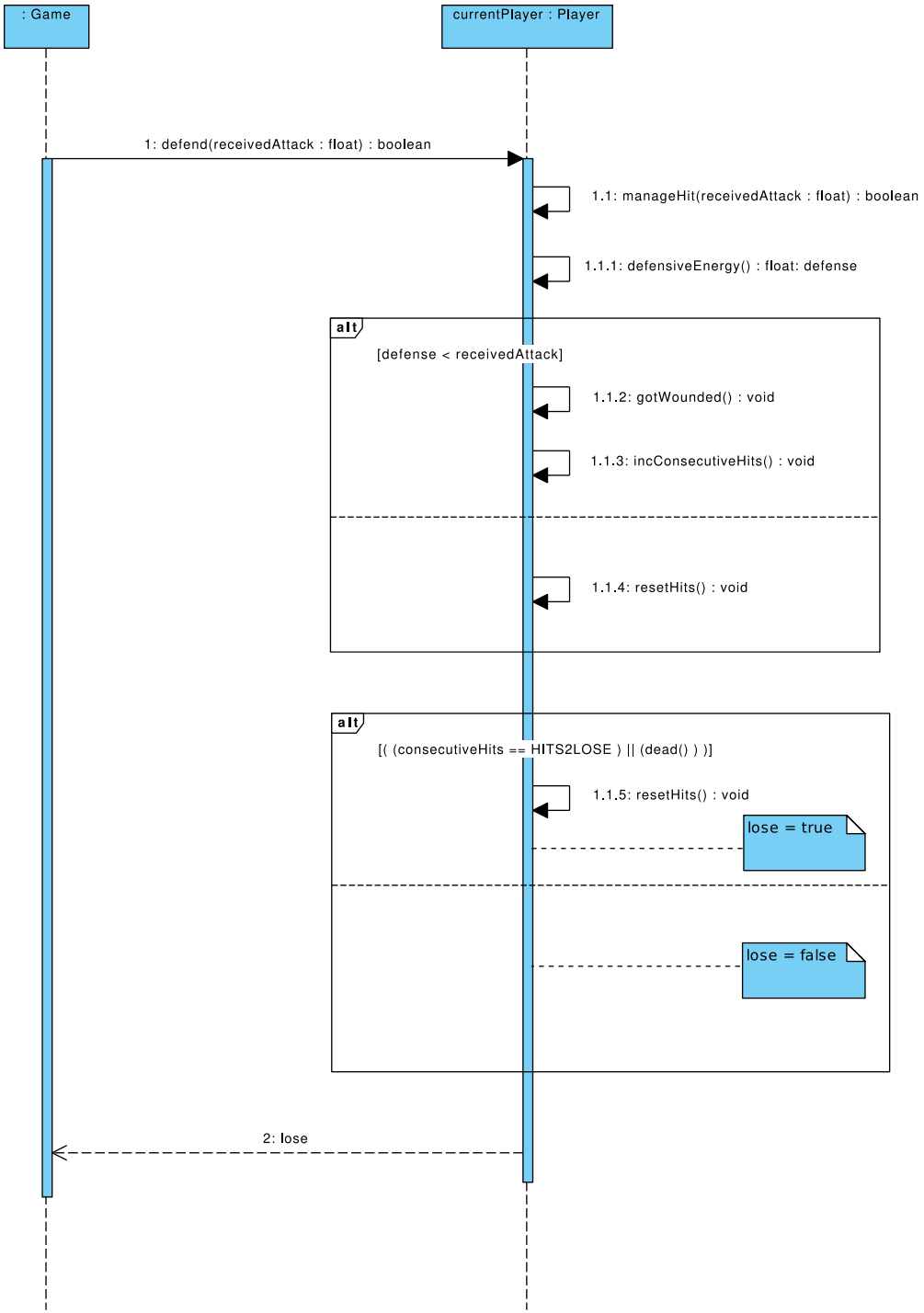

: Labyrinth
players : Player
labyrinth : char
monsters : Monster
player : Player
1: putPlayer2D(oldRow : int, oldCol : int, row : int, col : int, player : Player) : Monster
output = null
opt
[canStepOn(row,col)]
opt
[posOK(oldRow,oldCol)]
1.1: get(oldRow,oldCol): Player: p
opt
[p == player]
1.2: updateOldPos(row : int = oldRow, col : int = oldCol) : void
1.3: set(oldRow,oldCol,null)
1.4: monsterPos(row : int = row, col : int = col) : boolean: monsterPos
output = monster[row][col]
alt
[monsterPos]
1.5: set(row,col,COMBAT_CHAR)
1.6: get(row,col): : Monster: output
1.7: getNumber() : char: number
1.8: set(row,col,number)
1.9: set(row,col,player)
1.10: setPos(row : int = row, col : int = col) : void
return output

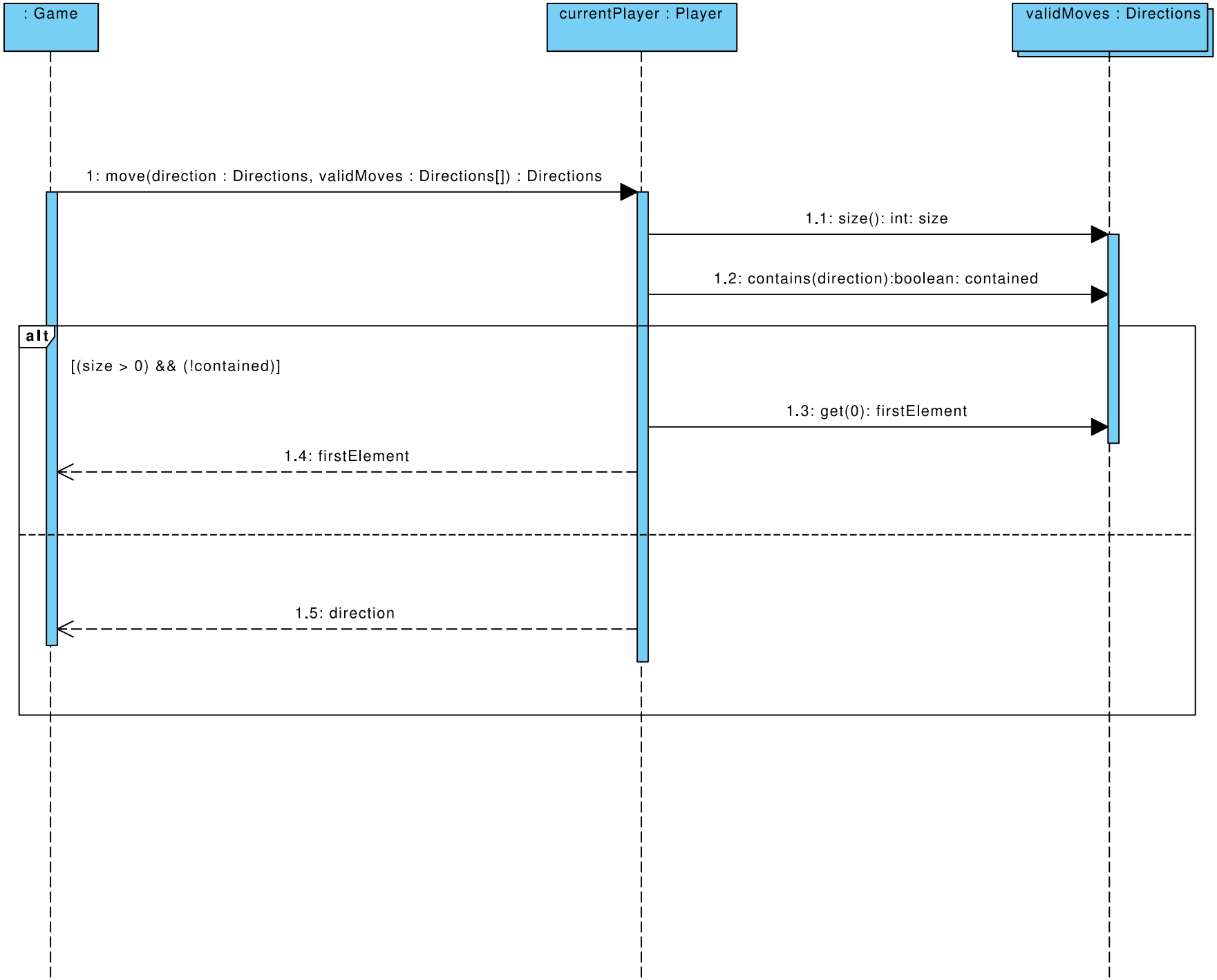

: Game
: Labyrinth
player : Player
1: putPlayer(direction : Directions, player : Player) : Monster
1.1: getRow() : int: oldRow
1.2: getCol() : int: oldCol
1.3: dir2Pos(row : int = oldRow, col : int = oldCol, direction : Directions = direction) : int[]: newPos
1.4: putPlayer2D(oldRow : int = oldRow, oldCol : int = oldCol, row : int = newPos[ROW], col : int = newPos[COL], player : Player = player) : Monster: monster
2: monster

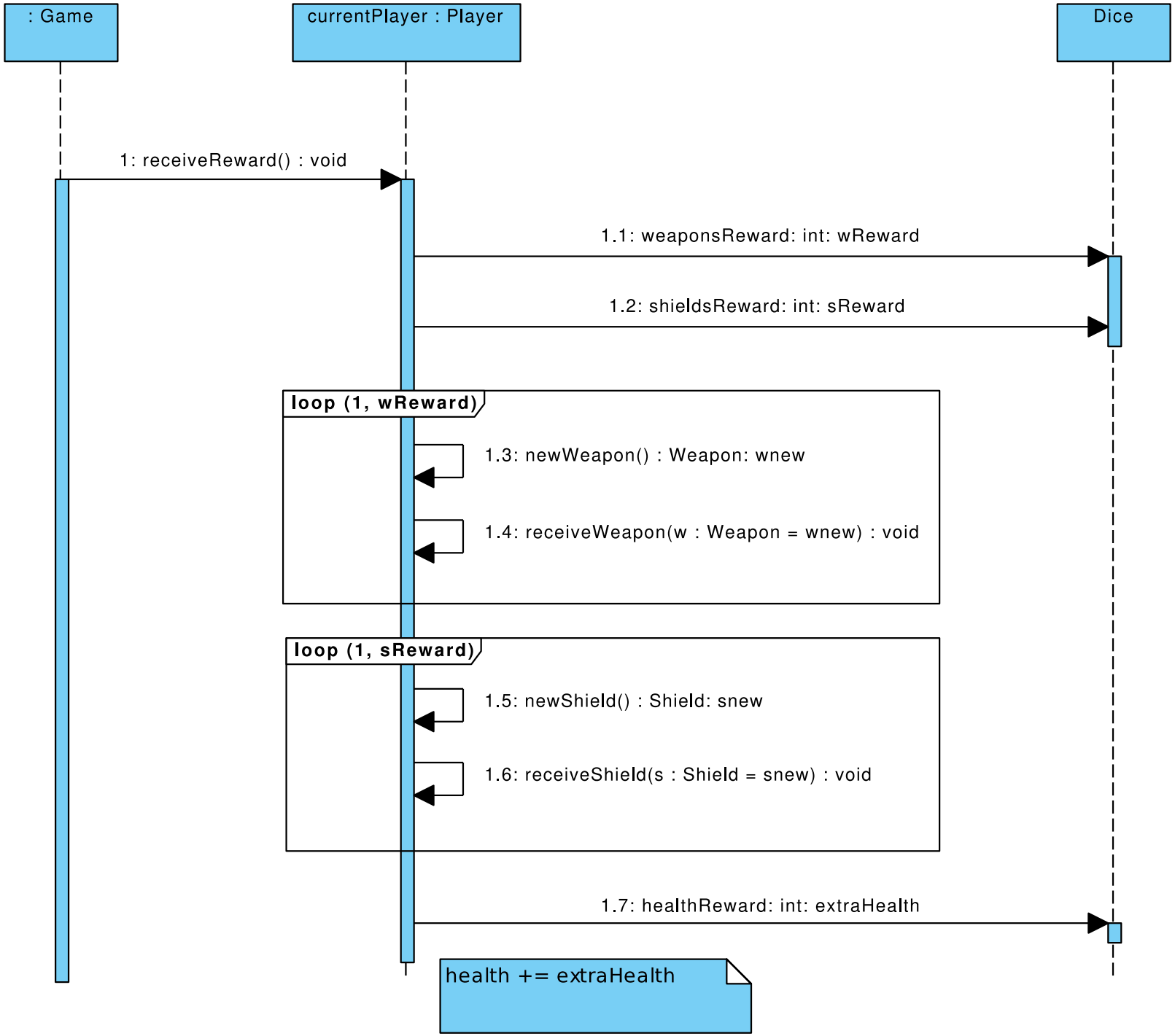

: Labyrinth
players : Player
1: spreadPlayers(players : Player[]) : void
loop
[for all]
1.1: next(): Player: p
1.2: randomEmptyPos() : int[]: pos
1.3: putPlayer2D(oldRow : int = -1, oldCol : int = -1, row : int = pos[ROW], col : int = pos[COL], player : Player = p) : Monster

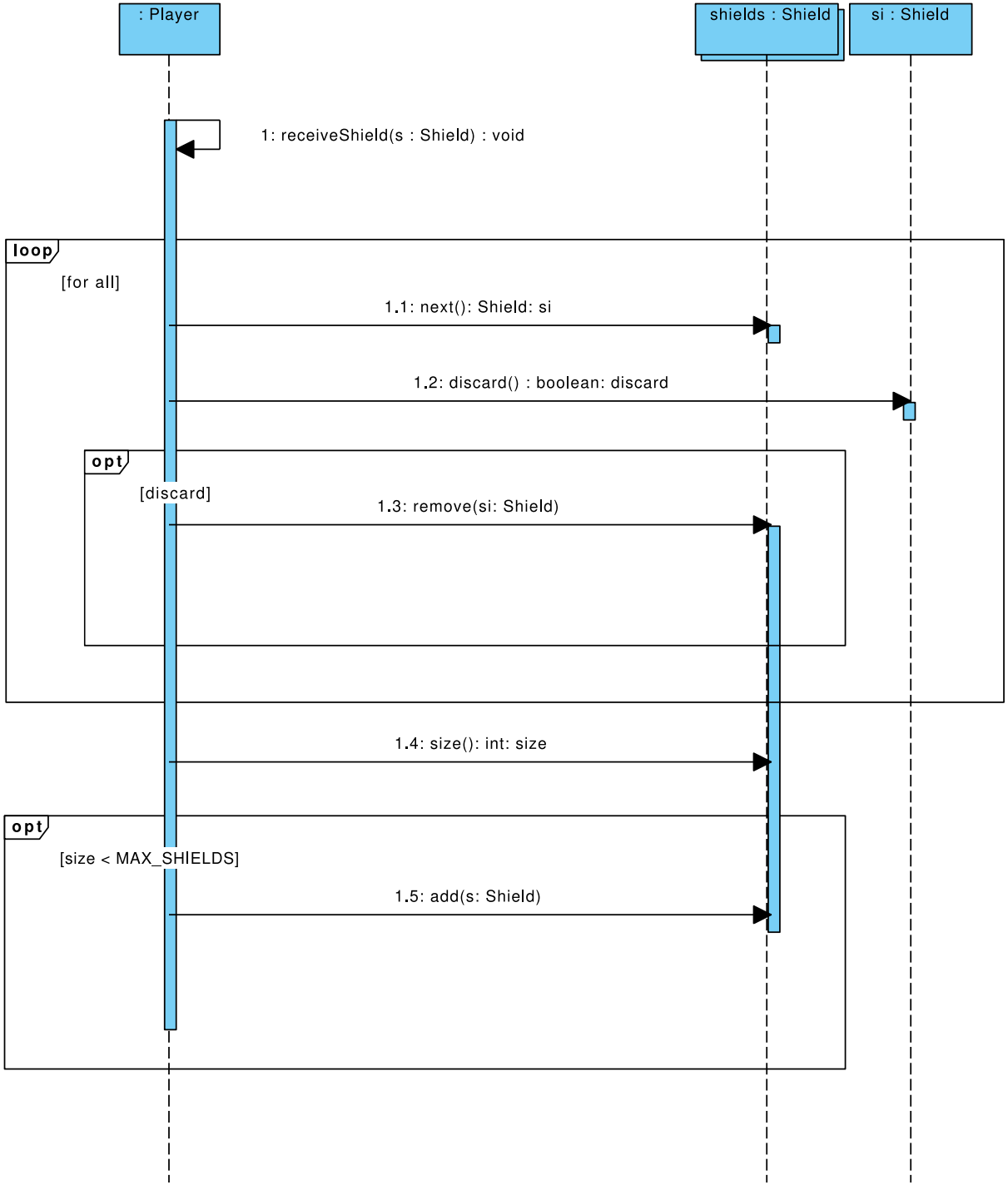

: Game
: Labyrinth
1: validMoves(row : int, col : int) : Directions[]
1.1: new
output : Directions
opt
[canStepOn(row+1, col)]
1.2: add(Directions.DOWN)
opt
[(canStepOn(row-1, col))]
1.3: add(Directions.UP)
opt
[(canStepOn(row, col+1))]
1.4: add(Directions.RIGHT)
opt
[(canStepOn(row, col-1))]
1.5: add(Directions.LEFT)
1.6: output

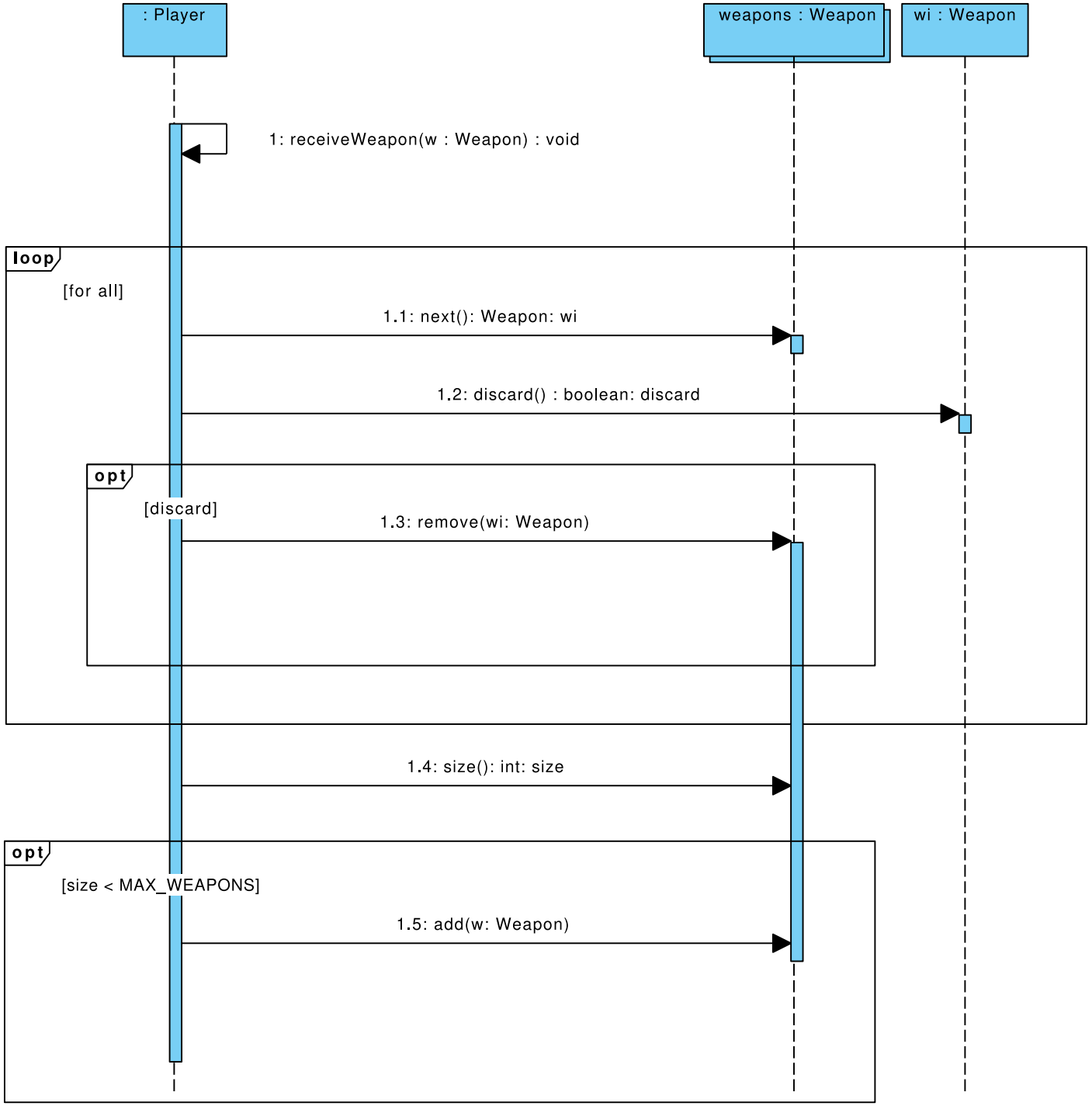

: Game
: Monster
Dice
1: defend(receivedAttack : float) : boolean
1.1: dead() : boolean: isDead
opt
[!isDead]
1.2: intensity(intelligence): float: defensiveEnergy
opt
[defensiveEnergy < receivedAttack]
1.3: gotWounded() : void
1.4: dead() : boolean: isDead
1.5: isDead


: Game
currentPlayer : Player
1: defend(receivedAttack : float) : boolean
1.1: manageHit(receivedAttack : float) : boolean
1.1.1: defensiveEnergy() : float: defense
alt
[defense < receivedAttack]
1.1.2: gotWounded() : void
1.1.3: incConsecutiveHits() : void
1.1.4: resetHits() : void
alt
[((consecutiveHits == HITS2LOSE) || (dead()))]
1.1.5: resetHits() : void
lose = true
lose = false
2: lose


: Game
currentPlayer : Player
validMoves : Directions
1: move(direction : Directions, validMoves : Directions[]) : Directions
1.1: size(): int: size
1.2: contains(direction):boolean: contained
alt
[(size > 0) && (!contained)]
1.3: get(0): firstElement
1.4: firstElement
1.5: direction


: Game
currentPlayer : Player
Dice
1: receiveReward() : void
1.1: weaponsReward: int: wReward
1.2: shieldsReward: int: sReward
loop (1, wReward)
1.3: newWeapon() : Weapon: wnew
1.4: receiveWeapon(w : Weapon = wnew) : void
loop (1, sReward)
1.5: newShield() : Shield: snew
1.6: receiveShield(s : Shield = snew) : void
1.7: healthReward: int: extraHealth
health += extraHealth


: Player
shields : Shield
si : Shield
1: receiveShield(s : Shield) : void
loop
[for all]
1.1: next(): Shield: si
1.2: discard() : boolean: discard
opt
[discard]
1.3: remove(si: Shield)
1.4: size(): int: size
opt
[size < MAX_SHIELDS]
1.5: add(s: Shield)


: Player
weapons : Weapon
wi : Weapon
1: receiveWeapon(w : Weapon) : void
loop
[for all]
1.1: next(): Weapon: wi
1.2: discard() : boolean: discard
opt
[discard]
1.3: remove(wi: Weapon)
1.4: size(): int: size
opt
[size < MAX_WEAPONS]
1.5: add(w: Weapon)